Assignment 1
AI-Assisted Cryptographic Vulnerability Discovery

Tareq Alsubaie *550258025*
Bharathwaj Murali *550086211*
Peisen Zheng *540660793*
Nishanth Shanmugasundaram *550258933*
Shyam Karthick Venkatraman Balakrishnan *550224271*

Abstract

This report analyses the Android application package (APK) a1_case1.apk and focuses on one core vulnerability related to randomness and authentication. Static decompilation shows that the application generates its session token inside the Login class using `java.util.Random`, a non-cryptographic pseudo-random number generator. This token is then stored in SharedPreferences and used to represent authenticated session state. Because the token has a security role but is generated using `java.util.Random`, which is not cryptographically secure, an attacker who reverse-engineers the application can analyse or reproduce the token generation process and undermine the integrity of the application's authentication state. Under a realistic local attacker model, this weakness can support session manipulation or impersonation. A secure fix is to replace `java.util.Random` with `SecureRandom` and avoid relying on locally predictable client-generated session state. GitHub Link

1 Introduction

An APK is the installable package format used by Android applications. Since APKs normally contain compiled bytecode rather than readable source code, decompilation is necessary when auditing application logic, authentication behaviour, and security-sensitive value generation. The APK package name and manifest structure were inspected after decompilation to identify the main entry activity and authentication-related classes.

The application is a simple local login demo with three screens: MainActivity, Login, and Profile. MainActivity accepts user credentials, Login verifies them and creates a session token, and Profile represents the post-login state. There is no backend service, so authentication and session handling are implemented locally. The goal of this analysis is to identify security-sensitive values treated as random within the application and determine whether the randomness mechanism used is appropriate for its security role.

2 System/Threat model

The system model contains four relevant components. First, MainActivity collects user credentials. Second, Login verifies the credentials and generates a session token. Third, SharedPreferences stores the session token as the application's local session state. Fourth, Profile represents the logged-in area and clears the token on logout.

The main protected assets are the integrity of the authentication flow, the confidentiality and integrity of the session token, and the correctness of the application's authenticated state. The important data flow is: user input is received in MainActivity, credentials are checked in Login, Login.generateSessionToken() creates the token, and SharedPreferences stores that token for later use by the application.

The attacker in this model has the following capabilities:

- reverse engineer the APK using decompilation tools;
- inspect application logic and identify how session tokens are generated;
- access or modify local storage on a rooted device or through privileged malware;
- observe the format and structure of the session token used by the application.

This is a realistic local attacker, such as a rooted-device user, forensic examiner, or privileged malware instance. The attacker's goal is to undermine session integrity by predicting, reproducing, or forging the value used to represent authenticated state. Because the attacker can inspect the token generation logic after reverse engineering the APK, the predictability of `java.util.Random` becomes exploitable, allowing the attacker to reason about the space of possible session tokens.

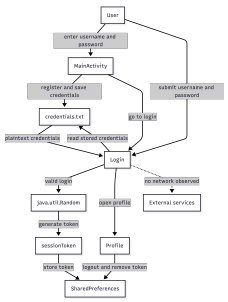

Figure 1: Application System Model

3 Vulnerability Discovery & Explanation

3.1 How you found the vulnerability

The vulnerability was found by statically decompiling the APK and tracing the authentication flow across MainActivity, Login, and Profile. AI assistance was used to suggest appropriate APK decompilation tools and to help evaluate whether the random API used in the code was suitable for generating security-sensitive values. All conclusions were validated manually against the decompiled source.

During static analysis, all values treated as tokens, codes, or identifiers were inspected. The method `generateSessionToken()` in the Login class was identified as the only location where randomness is used for a security-sensitive value. This method uses the API `java.util.Random` to construct an authentication token. Because the token is later stored in SharedPreferences and represents logged-in session state, it was selected as the application's core randomness-related vulnerability.

3.2 Vulnerability details

The core vulnerability is insecure session token generation using weak randomness. The exact location is the Login class, in the method `generateSessionToken()`. That method creates a 16-character token from an alphanumeric alphabet using `java.util.Random` and then stores the result in SharedPreferences as the application's session state.

This is a security problem because the generated value is not a cosmetic or UI-only random string. It has a security role: it represents authenticated state. Session tokens must therefore be generated with strong unpredictability so that attackers cannot infer or reproduce them. However, `java.util.Random` is a general-purpose pseudo-random number generator, not a cryptographically secure one. It is not designed for security-sensitive values such as session identifiers.

A realistic attack scenario is as follows:

- The attacker decompiles the APK using a tool such as JADX and inspects the Login class.
- The attacker observes that `generateSessionToken()` creates the authentication token using `java.util.Random`.
- Because the token format and generation algorithm are visible in the decompiled code, the attacker understands how tokens are constructed.
- The attacker gains local access to the device, for example through rooting, malware, or forensic access, and can inspect or modify the application's SharedPreferences.
- Knowing the token format and generation process, the attacker can reproduce candidate tokens more easily than if a cryptographically secure generator were used.
- The attacker can then manipulate or inject a token into the application's stored session state and impersonate an authenticated user.

The weakness is therefore not simply that the token "looks random." The real problem is that the application assigns a security role to a value generated by a non-cryptographic API. The design assumes unpredictability, but the chosen generator does not provide the level of unpredictability required for authentication security.

4 Concrete fix/mitigation and why it works

The primary fix is to replace `java.util.Random` with `SecureRandom` in `Login.generateSessionToken()`. SecureRandom is designed for security-critical applications and draws entropy from operating system sources, making the resulting tokens far more resistant to prediction or reproduction by attackers. This directly improves the security property that the current design is missing: cryptographically strong unpredictability.

A stronger mitigation is also architectural. The application should avoid relying on locally generated and locally trusted client-side session values as the sole basis for authenticated state. If a token represents authentication, it should be generated and validated in a way that preserves integrity against local tampering. Even in a local-only demo application, replacing `java.util.Random` with `SecureRandom` is the minimum technically sound correction because it aligns the random generator with the token's security role. [1].

References

[1] ORACLE. Securerandom (java platform se), 2026. https://docs.oracle.com/javase/8/docs/api/java/security/SecureRandom.html, Accessed: 2026-03-08.